

1.5em

LEONARDO LIMA DIONIZIO

TCC LEONARDO DIONIZIO

(versão pré-defesa, compilada em 23 de novembro de 2023)

Trabalho apresentado como requisito parcial à conclusão do Curso de Bacharelado em Ciência da Computação, Setor de Ciências Exatas, da Universidade Federal do Paraná.

Área de concentração: *Ciência da Computação*.

Orientador: Eduardo J. Spinosa.

CURITIBA PR

2023

RESUMO

O resumo deve conter no máximo 500 palavras, devendo ser justificado na largura da página e escrito em um único parágrafo¹ com um afastamento de 1,27 cm na primeira linha. O espaçamento entre linhas deve ser de 1,5 linhas. O resumo deve ser informativo, ou seja, é a condensação do conteúdo e expõe finalidades, metodologia, resultados e conclusões.

Suspendisse vitae elit. Aliquam arcu neque, ornare in, ullamcorper quis, commodo eu, libero. Fusce sagittis erat at erat tristique mollis. Maecenas sapien libero, molestie et, lobortis in, sodales eget, dui. Morbi ultrices rutrum lorem. Nam elementum ullamcorper leo. Morbi dui. Aliquam sagittis. Nunc placerat. Pellentesque tristique sodales est. Maecenas imperdiet lacinia velit. Cras non urna. Morbi eros pede, suscipit ac, varius vel, egestas non, eros. Praesent malesuada, diam id pretium elementum, eros sem dictum tortor, vel consectetur odio sem sed wisi.

Sed feugiat. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Ut pellentesque augue sed urna. Vestibulum diam eros, fringilla et, consectetur eu, nonummy id, sapien. Nullam at lectus. In sagittis ultrices mauris. Curabitur malesuada erat sit amet massa. Fusce blandit. Aliquam erat volutpat. Aliquam euismod. Aenean vel lectus. Nunc imperdiet justo nec dolor.

Etiam euismod. Fusce facilisis lacinia dui. Suspendisse potenti. In mi erat, cursus id, nonummy sed, ullamcorper eget, sapien. Praesent pretium, magna in eleifend egestas, pede pede pretium lorem, quis consectetur tortor sapien facilisis magna. Mauris quis magna varius nulla scelerisque imperdiet. Aliquam non quam. Aliquam porttitor quam a lacus. Praesent vel arcu ut tortor cursus volutpat. In vitae pede quis diam bibendum placerat. Fusce elementum convallis neque. Sed dolor orci, scelerisque ac, dapibus nec, ultricies ut, mi. Duis nec dui quis leo sagittis commodo.

Aliquam lectus. Vivamus leo. Quisque ornare tellus ullamcorper nulla. Mauris porttitor pharetra tortor. Sed fringilla justo sed mauris. Mauris tellus. Sed non leo. Nullam elementum, magna in cursus sodales, augue est scelerisque sapien, venenatis congue nulla arcu et pede. Ut suscipit enim vel sapien. Donec congue. Maecenas urna mi, suscipit in, placerat ut, vestibulum ut, massa. Fusce ultrices nulla et nisl.

Palavras-chave: Palavra-chave 1. Palavra-chave 2. Palavra-chave 3.

¹E também não deve ter notas de rodapé; em outras palavras, não siga este exemplo... ;-)

ABSTRACT

The abstract should be the English translation of the “resumo”, no more, no less.

Suspendisse vitae elit. Aliquam arcu neque, ornare in, ullamcorper quis, commodo eu, libero. Fusce sagittis erat at erat tristique mollis. Maecenas sapien libero, molestie et, lobortis in, sodales eget, dui. Morbi ultrices rutrum lorem. Nam elementum ullamcorper leo. Morbi dui. Aliquam sagittis. Nunc placerat. Pellentesque tristique sodales est. Maecenas imperdiet lacinia velit. Cras non urna. Morbi eros pede, suscipit ac, varius vel, egestas non, eros. Praesent malesuada, diam id pretium elementum, eros sem dictum tortor, vel consectetur odio sem sed wisi.

Sed feugiat. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Ut pellentesque augue sed urna. Vestibulum diam eros, fringilla et, consectetur eu, nonummy id, sapien. Nullam at lectus. In sagittis ultrices mauris. Curabitur malesuada erat sit amet massa. Fusce blandit. Aliquam erat volutpat. Aliquam euismod. Aenean vel lectus. Nunc imperdiet justo nec dolor.

Etiam euismod. Fusce facilisis lacinia dui. Suspendisse potenti. In mi erat, cursus id, nonummy sed, ullamcorper eget, sapien. Praesent pretium, magna in eleifend egestas, pede pede pretium lorem, quis consectetur tortor sapien facilisis magna. Mauris quis magna varius nulla scelerisque imperdiet. Aliquam non quam. Aliquam porttitor quam a lacus. Praesent vel arcu ut tortor cursus volutpat. In vitae pede quis diam bibendum placerat. Fusce elementum convallis neque. Sed dolor orci, scelerisque ac, dapibus nec, ultricies ut, mi. Duis nec dui quis leo sagittis commodo.

Aliquam lectus. Vivamus leo. Quisque ornare tellus ullamcorper nulla. Mauris porttitor pharetra tortor. Sed fringilla justo sed mauris. Mauris tellus. Sed non leo. Nullam elementum, magna in cursus sodales, augue est scelerisque sapien, venenatis congue nulla arcu et pede. Ut suscipit enim vel sapien. Donec congue. Maecenas urna mi, suscipit in, placerat ut, vestibulum ut, massa. Fusce ultrices nulla et nisl.

Keywords: Keyword 1. Keyword 2. Keyword 3.

LISTA DE FIGURAS

1.1	Gráfico que faz um levantamento de técnicas usadas para geração de conteúdo em jogos, onde AL equivale a <i>Adversarial Learning</i> . Fonte: (Liu et al., 2020) . . .	11
2.1	Representação dos elementos do jogo <i>Super Mario Bros</i> pela VGLC e pelos autores de (Volz et al., 2018), respectivamente. Fonte: (Volz et al., 2018)	17
2.2	Representação da arquitetura da GAN utilizada no paper. Fonte: (Volz et al., 2018)	18
2.3	Representação do processo de treinamento e de LVE. Fonte: (Volz et al., 2018) .	19
2.4	Nível gerado pelos autores, com progressão de dificuldade no eixo x. Fonte: (Volz et al., 2018)	19
2.5	Nível gerado pelos autores, de acordo com a performance do agente. A letra a utiliza F_3 , enquanto a parte b utiliza F_4 . Fonte: (Volz et al., 2018)	20
3.1	Fases geradas pela CESAGAN. Em a, níveis jogáveis e em b níveis injogáveis. Fonte: (Torrado et al., 2019)	22
5.1	Fases aleatórias geradas pela GAN treinada com o mundo 2.	31
5.2	Fases geradas pela GAN treinada com o mundo 2 utilizando o CMA-ES como técnica de LVE, maximizando canos	31
5.3	Fases aleatórias geradas pela GAN treinada com o mundo 5.	32
5.4	Fases geradas pela GAN treinada com o mundo 5 utilizando o CMA-ES como técnica de LVE, maximizando inimigos	33
5.5	Fases geradas pela GAN treinada com o mundo 5 utilizando o PSO como técnica de LVE, maximizando inimigos	34
5.6	Fases geradas pela GAN treinada com o mundo 5 utilizando o NSGA-II como técnica de LVE, maximizando canos e inimigos simultaneamente.	35
5.7	Fases geradas pela GAN treinada com o mundo 6 utilizando o CMA-ES como técnica de LVE, maximizando inimigos	38
5.8	Classificação corretas das entradas aleatórias	39
5.9	Classificação corretas das entradas do CMA-ES	39
5.10	Classificação corretas das entradas do PSO	40

LISTA DE TABELAS

4.1	Número de Níveis por Mundo	24
4.2	Configurações dos Algoritmos com seus Hiperparâmetros.	25
4.3	Resultados de Precisão, Recall e F1-Score por Mundo no teste	27
5.1	Comparação de Tempo e Fitness do CMA-ES e PSO	28
5.2	No Caption	29
5.3	Estatísticas dos conjuntos de treino maximizando o número de canos	36
5.4	Estatísticas dos conjuntos de treino maximizando o número de inimigos	37
5.5	Descrição do Conjunto de Treino	41

LISTA DE ACRÔNIMOS

GAN	<i>Generative Adversarial Network</i>
WGAN	<i>Wassertein Generative Adversarial Network</i>
PCGML	<i>Procedural Content Generation using Machine Learning LVE</i>
<i>Latent Variable</i>	
<i>Evolution</i>	
CMA-ES	<i>Covariance Matrix Adaptation Evolutionary Strategy</i>
PSO	<i>Particle Swarm Optimization</i>
NSGA	<i>Non-Dominated Sorting Genetic Algorithm</i>
MCDMA	<i>Multiple Criteria Decision Making</i>
CP	<i>Compromise Programming</i>

LISTA DE SÍMBOLOS

α	alfa, primeira letra do alfabeto grego
β	beta, segunda letra do alfabeto grego
γ	gama, terceira letra do alfabeto grego
ω	ômega, última letra do alfabeto grego
π	pi
τ	Tempo de resposta do sistema
θ	Ângulo de incidência do raio luminoso

SUMÁRIO

1	INTRODUÇÃO	11
1.1	OBJETIVO	12
2	FUNDAMENTAÇÃO TEÓRICA.	13
2.1	REDES NEURAIS ADVERSÁRIAS GENERATIVAS.	13
2.2	CMA-ES	14
2.3	EVOLUÇÃO DE VARIÁVEL LATENTE (LVE)	14
2.4	GERAÇÃO DE CONTEÚDO PROCEDURAL USANDO APRENDIZADO DE MÁQUINA (PCGML)	15
2.5	NSGA-II.	15
2.5.1	Decisão entre soluções	16
2.6	PSO	16
2.7	ALGORITMO A ESTRELA	16
2.8	MARIOGAN	17
2.9	CONCLUSÃO	20
3	TRABALHOS RELACIONADOS	21
3.1	GERAÇÃO DE CONTEÚDO PROCEDURAL VIA MACHINE LEARNING UTILIZANDO REDES GENERATIVAS ADVERSÁRIAS	21
3.2	CONCLUSÃO	22
4	PROPOSTA	24
4.1	CONJUNTO DE TREINO	24
4.2	ARQUITETURA DO MODELO NEURAL	24
4.3	MÉTODOS PARA LVE	25
4.4	FUNÇÕES OBJETIVOS	25
4.5	AVALIAÇÃO DE CONTEÚDO	26
4.5.1	Classificação de níveis	26
4.5.2	Jogabilidade e performance de jogo	27
4.6	CONCLUSÃO	27
5	RESULTADOS.	28
5.1	COMPARAÇÃO DE TEMPO E PERFORMANCE ENTRE CMA-ES E PSO	28
5.2	NÍVEIS EM PROGRESSÃO DE DIFICULDADE.	29
5.3	TOPOLOGIA DOS NÍVEIS	29
5.3.1	Análise visual	30
5.4	CLASSIFICAÇÃO DOS PADRÕES GERADOS	31
5.5	JOGABILIDADE	34

6	CONCLUSÃO	42
	REFERÊNCIAS	43

1 INTRODUÇÃO

No início da adesão aos modelos neurais, sejam eles profundos ou perceptrons, tais modelos se popularizaram inicialmente nas tarefas de regressão e classificação, gerando competições populares como a ImageNet, competição em que o intuito é classificar imagens corretamente dentre 1000 classes. Ainda assim, aproximar uma distribuição só tornou-se possível com o surgimento das Redes Neurais Adversariais (GANs), inicialmente propostas em (Goodfellow et al., 2014). O paradigma do aprendizado adversarial permitiu, através do uso de uma rede geradora e uma rede discriminadora, que uma rede neural profunda se aproxime de uma distribuição de forma a, ao invés de tomar uma decisão em relação a sua entrada, gerar conteúdo que se assemelhe a um conjunto conhecido.

Em (Goodfellow et al., 2014), o intuito desta geração de conteúdo era a geração de imagens. Com o refinamento, este modelo tornou-se popular em outras tarefas, como a transferência de estilo proposta em (Zhu et al., 2020) e a tradução de texto para imagem ilustrada em (Reed et al., 2016). Embora as redes adversariais generativas tenham tornado-se bastante conhecida nestas tarefas relacionadas com imagens, suas aplicações vão muito além destas. Uma das indústrias que muito se beneficia da geração de conteúdo é o ramo dos videogames. Embora a geração de conteúdo em jogos esteja presente desde o jogo de 1984 *Elite*, GANS são muito utilizadas na geração para jogos, principalmente para níveis, conforme a **Figura 1.1**.

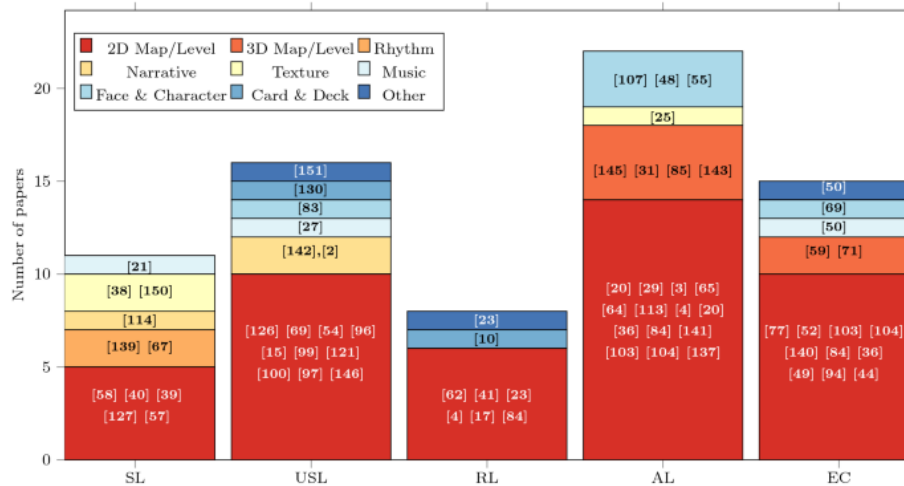


Figura 1.1: Gráfico que faz um levantamento de técnicas usadas para geração de conteúdo em jogos, onde AL equivale a *Adversarial Learning*. Fonte: (Liu et al., 2020)

Grandes exemplos do uso de geração procedural são os jogos *Minecraft*, onde cada mapa é gerado em cima de um número aleatório, uma seed, tornando cada mundo diferente do outro - de forma tão diversificada que seria impensável um humano fazer isso -, *No Man's Sky* e o recente *Starfield*, onde planetas e biomas são gerados proceduralmente num ecossistema online multijogador, e jogos do gênero rogue, onde cada vez que o jogador morre pode melhorar sua

personagem e mais uma vez enfrentar os desafios do início, porém, diferentes a cada jogada, como *Rogue Legacy* (1 e 2), *Enter The Gungeon* e *Hades*.

A área acadêmica também estuda a geração de conteúdo para jogos, muitos destes estudos ligados a geração de níveis através do aprendizado adversarial, conforme levantado em (Liu et al., 2020) na **Figura 1.1**. Um popular modelo para geração de níveis para o jogo *Super Mario Bros.* foi proposto em (Volz et al., 2018). A grande inovação do artigo que o tornou popular no nicho não foi a arquitetura da GAN em si, mas sim o fato de utilizar o conceito de evolução de variável latente (LVE) para manipular as entradas do gerador de forma que o conteúdo produzido pelo modelo seguisse certo padrão através de um algoritmo genético, o CMA-ES, proposto inicialmente em (Hansen et al., 2003). A demonstração do artigo foi treinando a rede neural proposta apenas com uma única fase e avaliando a qualidade da fase com a função objetivo do algoritmo genético e também através de métricas de um agente A estrela jogando o nível gerado, porém não se preocupa em verificar se estas fases seguem ou não de forma suficiente o conjunto de fases conhecidas. Outra limitação de (Volz et al., 2018) é o uso apenas do CMA-ES, não se preocupando em outras formas de explorar o espaço latente, ou tentar uma abordagem multi-objetivo, algo sugerido no final do artigo como possível estudo futuro.

1.1 OBJETIVO

Este trabalho utiliza a rede geradora proposta em (Volz et al., 2018) e, além da já treinada rede com uma única fase do jogo disponibilizada pelos autores, neste estudo o modelo defendido do artigo também foi treinado com as fases de cada um dos 8 mundos do jogo *Super Mario Bros* (agrupamentos de até três níveis, que tem como objetivo um aumento de dificuldade gradual) no intuito de avaliar tanto a qualidade, jogabilidade e diversidade das fases geradas em um conjunto maior de níveis quanto se as redes aproximam ou não de forma diferenciável o conjunto de fases apresentados no treinamento do modelo gerador.

Para além disso, avaliamos a efetividade do algoritmo proposto para a evolução de variáveis latentes, o CMA-ES, em comparação com um algoritmo simples de otimização de partículas (PSO). Também comparamos uma abordagem multi-objetivo através do algoritmo NSGA-II proposto em (Deb, 2002) otimizando diferentes funções tanto com relação a topologia do nível quanto as propostas em (Volz et al., 2018).

2 FUNDAMENTAÇÃO TEÓRICA

Este capítulo revisita os tópicos fundamentais deste trabalho, incluindo os princípios de redes neurais adversariais, evolução de variável latente e algoritmos genéticos para exploração do espaço latente com o exemplo do CMA-ES; explora a literatura no tópico de geração de conteúdo procedural utilizando aprendizado de máquina; e apresenta o principal artigo que guiou este trabalho, que utiliza uma DCGAN para gerar níveis do jogo Super Mario Bros.(1985).

2.1 REDES NEURAS ADVERSARIAIS GENERATIVAS

O paradigma do Aprendizado Adversarial e as Redes Neurais Adversariais Generativas (GANs) foram introduzidos em (Goodfellow et al., 2014) em 2014 como solução para o problema de estimação de máxima verossimilhança, dilema no qual o custo computacional e a complexidade são extremamente altos, podendo chegar a ser computacionalmente inviável a depender do contexto. Enquanto a gama mais popular de modelos profundos na época de publicação do artigo consistiam de apenas uma rede neural de alta profundidade usada em classificação e, em certos casos, regressão, o aprendizado adversarial proposto pelo artigo usufrui de duas MLPs.

Um desses modelos é apelidado de gerador, abreviado como G , enquanto o outro modelo é chamado de discriminador e abreviado como D . O modelo gerador representa uma função diferenciável responsável por aproximar a distribuição de seus dados de treinamento. Para gerar novos dados, a rede G projeta entradas ruidosas no espaço dos dados conhecidos, na intenção de aproximar o conteúdo gerado da distribuição dos conjuntos conhecidos.

Por sua vez, a MLP D tem como objetivo traçar um plano que separe as distribuições do conteúdo original das distribuições das projeções de G . A saída da rede discriminadora, no artigo, é descrita como um único escalar que representa a probabilidade de uma entrada não ser gerada. Essencialmente, G e D tornam-se antagônicas a partir do momento que suas respectivas funções de perda colocam como objetivo penalizar a outra. Para que isso seja possível, G é treinada para minimizar $\log(1 - D(G(z)))$ enquanto D é treinada para maximizar a probabilidade de acertar se uma entrada é sintética ou real. Este treinamento antagônico leva essas duas redes a disputarem um jogo MinMax de dois jogadores. Alternando a otimização de D e G , a rede discriminadora fica mais perto da solução ótima, enquanto a MLP geradora muda aos poucos para sua distribuição aproximar cada vez mais da original e, por consequência, tornar a solução que o discriminador busca mais complexa de se aproximar.

2.2 CMA-ES

CMA-ES, a *Covariance Matrix Adaptation Evolution Strategy*, foi instanciada primeiro em (Hansen et al., 2003). É uma estratégia evolucionária que se comporta bem com vetores de números reais, que neste casos são as entradas do gerador, e sua principal característica é o uso de uma matriz de covariância estimada iterativamente por diferenças finitas.

Como está no seu nome, o algoritmo faz parte de um grupo de algoritmos evolucionários chamados *evolutionary strategies*, que é um grupo de algoritmos caixa preta caracterizados também pelo uso da ordenação por ranking. A CMA-ES tem se mostrado eficiente para otimizar problemas não-convexos e não-lineares em domínios contínuos.

O que este algoritmo faz é calcular uma gaussiana com base na matriz de covariância estimada. Esta gaussiana é traduzida numa elipse que se desloca de forma a contemplar as melhores soluções. Quanto mais distantes as melhores soluções estão do domínio dessa elipse, maior o *step size* dela, ou seja, seu raio. De certa forma, no começo, o *step size* costuma ser mais alto pois as melhores soluções conhecidas estão esparsas e a partir deste ponto de partida há a exploração global. Conforme a elipse fica melhor posicionada no domínio, as melhores soluções ficarão menos esparsas, fazendo com que o *step size* diminua proporcionalmente com a concentração da população.

Segue um pseudo-código do que seria o CMA-ES:

Algoritmo 1 CMA-ES

- 1: Inicialize a população: λ
 - 2: Inicialize o step size: σ
 - 3: Defina o tamanho da população: m
 - 4: defina o número de melhores candidados μ
 - 5: Defina os pesos: $W_1, W_2, \dots, W_m$
 - 6: **repeat**
 - 7: **for all** $x \in \lambda$ **do**
 - 8: Calcula a gaussiana para x
 - 9: **end for**
 - 10: Ordena λ pelo fitness
 - 11: $c \leftarrow \text{calcula}_{\text{covariancia}}(\lambda)$
 - 12: $m \leftarrow \sum_{i=1}^{\mu} W_i \cdot X_i$
 - 13: $\sigma \leftarrow \text{atualiza}(\sigma)$
 - 14: **until** número de iterações $> \text{max_iter}$
-

2.3 EVOLUÇÃO DE VARIÁVEL LATENTE (LVE)

LVE - *Latent Variable Evolution* - surgiu primeiro para buscar digitais específicas num certo espaço mapeado por uma GAN em (Bontrager et al., 2017). Como dito anteriormente, a rede neural geradora de uma GAN mapeia entradas ruidosas para um espaço. Estas entradas

ruidosas também são chamadas de variáveis latentes, ou seja, variáveis que são difíceis ou impossível de controlar e interpretar antes de um resultado concreto.

A partir disso, a técnica de LVE tem como ideia manipular as entradas ruidosas de um modelo para que sua saída se encaixe dentro de certas especificações, ou seja, adiciona ao conteúdo gerado restrições que até o momento atual não são controláveis a nível de treinamento, pois uma rede geradora tem como única função reproduzir a distribuição de seus dados de teste. Em (Bontrager et al., 2017), a técnica de LVE é utilizada para a quebra de identificação de digitais. A técnica consiste de duas etapas: primeiro, treinar uma rede neural para gerar o conteúdo alvo e, após isso, usar um algoritmo para buscar no espaço latente uma entrada que gere uma saída dentro das condições especificadas.

O algoritmo utilizado por (Bontrager et al., 2017) para busca no espaço latente é o já explicado anteriormente CMA-ES. A justificativa do uso do CMA-ES pelo autores é que a função objetivo é não contínua e, pela natureza de redes convolucionais, também não é linearmente separável.

2.4 GERAÇÃO DE CONTEÚDO PROCEDURAL USANDO APRENDIZADO DE MÁQUINA (PCGML)

A survey de Summerville e colegas (Summerville et al., 2018) define *Procedural Content Generation Via Machine Learning* como "geração de conteúdo de jogos usando modelos de *machine learning* em conteúdos existentes". Na área, busca-se aumentar o valor de replay, salvar memória, reduzir o custo e o esforço de produção, diversificar estética e reduzir a presença humana na geração de conteúdo de jogos. Na área acadêmica, além disso, pode-se dizer que se explora novas maneiras de experienciar os jogos, além de ser um desafio que estimula a criatividade computacional e o entendimento de *game design* pela construção de modelos.

2.5 NSGA-II

Em Abril de 2002 foi proposto a evolução do *nondominated sorting genetic algorithm* (algoritmo genético de ordenção não-dominada), o NSGA-II (Deb, 2002), que hoje em dia conta com 37971 citações (Connected Papers, acesso em 05/11/2023). Esse algoritmo foi proposto na intenção de contornar os três principais problemas dos algoritmos evolucionários apontados na época:

- Alta complexidade computacional (especificamente $O(MN^3)$, onde M é o número de objetivos e N o tamanho da população);
- A falta de elitismo;
- A necessidade de especificar parâmetros, como o caso do NSGA proposto em (?);

E sua implementação nos permite:

- Preservar a diversidade;
- e enfatizar soluções não-dominadas.

Tendo populações de tamanho N , o NSGA-II recombina tal população numa população de tamanho $2N$ e a ordena baseado no ranking da fronteira de pareto, priorizando soluções da fronteira classe 1 e completando com as fronteiras subsequentes. Quando a fronteira F_g não cabe mais inteira na próxima população, são selecionadas os indivíduos com maior crowd distance, que é a área/volume (dependente do tamanho do espaço objetivo Z do problema) ao seu redor sem nenhuma solução presente (cabe enfatizar que as funções objetivo são normalizadas para que nenhuma domine a outra). Este operador permite escolher as soluções que estão mais distantes das outras, por consequência também sendo um preservador de diversidade por si só.

2.5.1 Decisão entre soluções

Um dos dilemas de algoritmos multi-objetivo é projetado numa área chamada de MCDM (*multiple criteria decision making*, em que deve-se decidir entre as soluções qual é a melhor. Os experimentos deste trabalho seguem a ideia de (Rao, 2010) de compromise programming (CP) para este propósito conforme discutido em (Ringuest, 1992).

Assumindo a existência de uma solução ideal e a conversão próxima a fronteira de pareto, o suposto ponto ideal é projetado levando em conta as soluções não dominadas existentes (ou seja, os menores valores dos objetivos). Uma vez feita essa projeção, o ponto escolhido é o de menor distância deste ponto.

2.6 PSO

Otimização por enxames de partículas foi proposta por (Kennedy e Eberhart, 1995). Segundo os autores, é um método de otimização para funções contínuas e não-lineares que simula um modelo social simplificado. O algoritmo instância uma população candidata e a move pelo domínio de acordo com fórmulas matemáticas simples em relação a posição e velocidade da partícula.

A metaheurística associada ao PSO faz pouca ou nenhuma suposição do problema que será otimizado. Outra vantagem é que o algoritmo não utiliza otimização por gradiente como muitos dos algoritmos de computação evolutiva, o que significa que a função objetivo não precisa ser diferenciável.

2.7 ALGORITMO A ESTRELA

O algoritmo A estrela é uma extensão do Algoritmo de Dijkstra que utiliza alguma metaheurística para guiar a busca pelo caminho mais curto ao objetivo. Para que o algoritmo funcione corretamente, a heurística definida para auxiliar a busca deve ser admissível, isto é, o

custo estimado para atingir o estado final em relação ao estado atual deve ser menor ou igual ao custo verdadeiro para atingir o estado final do estado atual.

A busca A estrela possui múltiplas aplicações, entre elas a busca por caminhos viáveis em videogames. Até os dias de hoje, o mais popular algoritmo para o teste de jogabilidade de níveis do jogo *Super Mario Bros* é o agente A* proposto por Robin Baumgarten em (Togelius et al., 2010).

2.8 MARIOGAN

Em maio de 2018 publicou-se o artigo que deu origem ao que outros entusiastas do estudo da PCGML chamam de MarioGAN (Volz et al., 2018). O artigo aborda o uso de GANs, mas não foca na arquitetura da rede geradora ou discriminadora, mas sim utiliza a ideia de LVE para geração de níveis procedurais no jogo *Super Mario Bros*. Aqui, seguindo a ideia de (Bontrager et al., 2017), o CMA-ES é utilizado para adicionar alguma restrição ao espaço latente do gerador, sendo testadas quatro funções-objetivo diferentes, duas delas relacionadas a disposição dos blocos da fase e outras duas em relação ao teste das fases pelo agente A* vencedor da competição Mario AI de 2009(Togelius et al., 2010) de autoria de Robin Baumgarten.

Para representar as fases, os autores utilizaram a representação do jogo *Super Mario Bros* disponível no repositório do *github* aberto da Video Game Level Corpus(VGLC) (Summerville et al., 2016), que, entre suas particularidades, simplificam a representação do jogo ao ignorar níveis específicos, onde a física do jogo diverge do padrão seguido pela maioria das fases, e também representa todos os inimigos da mesma maneira, tornando a tarefa possivelmente mais simples. Nesta representação, cada tipo de bloco é mapeado para um caracter específico, transformando a imagem em uma sequência de caracteres que indicam um bloco específico. Uma vez que o artigo implementa a arquitetura a partir da biblioteca *Pytorch*, esta representação textual precisou ser remapeada para uma representação numérica, conforme mostrado pela **Figura 2.1**.

Tile type	Symbol	Identity	Visualization
Solid/Ground	X	0	
Breakable	S	1	
Empty (passable)	-	2	
Full question block	?	3	
Empty question block	Q	4	
Enemy	E	5	
Top-left pipe	<	6	
Top-right pipe	>	7	
Left pipe	[	8	
Right pipe	]	9	

Figura 2.1: Representação dos elementos do jogo *Super Mario Bros* pela VGLC e pelos autores de (Volz et al., 2018), respectivamente. Fonte: (Volz et al., 2018)

A GAN convolucional profunda (DCGAN) utilizada no artigo é baseada na Wassertein Gan apresentada em (Arjovsky et al., 2017), em que o discriminador utiliza convoluções com deslocamento (*strided convolutions*), enquanto o gerador utiliza convoluções com deslocamento fracionário (*fractional-strided convolutions*). A arquitetura em (Volz et al., 2018) difere de (?) nos seguintes pontos:

- Tanto no discriminador quanto no gerador, é empregada uma camada de normalização de bloco (*batch normalization* após cada camada convolucional;
- A função de ativação de todas as camadas (inclusive na de saída) é a ReLU (O discriminador, como em (Arjovsky et al., 2017), utiliza a Leaky ReLU como função de ativação em suas camadas).

Tanto o discriminador quanto o gerador são treinados com o otimizador RMSprop, com um *batch size* de tamanho 32 e a taxa de aprendizado de 0,00005 por 5000 iterações. A arquitetura é devidamente ilustrada na **Figura 2.2**.

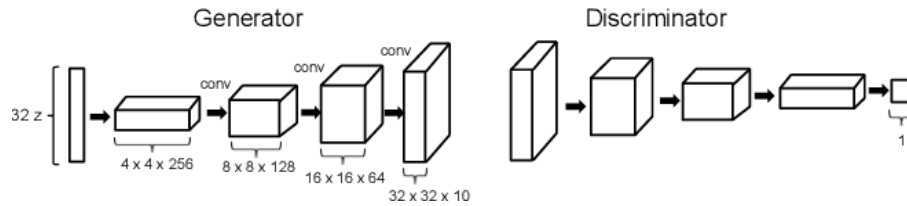


Figura 2.2: Representação da arquitetura da GAN utilizada no paper. Fonte: (Volz et al., 2018)

O gerador utilizado no artigo é treinado apenas com o primeiro nível do jogo, na intenção de mostrar como a LVE e a GAN treinada poderiam, em conjunto, gerar níveis diferentes e jogáveis. Outra escolha interessante do artigo da MarioGAN (Volz et al., 2018) é que, para gerar múltiplas amostras com apenas uma fase, a representação numérica foi separada com base em uma janela deslizante com dimensões de 14 de altura e 28 de largura, com passo de tamanho um.

Uma noção geral do processo executado por (Volz et al., 2018) está devidamente traduzido na **Figura 2.3**. Este método é dividido em duas fases básicas distintas: primeiro, a WGAN é treinada de maneira não-supervisionada para gerar níveis do *Super Mario Bros*; após isso, o CMA-ES é utilizado de forma a buscar no espaço latente da GAN, ou seja, alterar a entrada do modelo para satisfazer propriedades específicas. Embora o artigo se baseie apenas em níveis do jogo *Super Mario Bros*, os autores clamam que o modelo é suficientemente bom para generalizar qualquer jogo com representação compatível.

Quanto as funções minimizadas em relação a topologia do nível minimizadas no artigo, as equações (2.1) e (2.2) as traduzem, em que g e t são respectivamente a fração da quantidade de blocos de chão na ultima linha da matriz que representa a fase e a fração desejada, e n o número de inimigos.

$$F_1 = \sqrt{(g - t)^2} \quad (2.1)$$

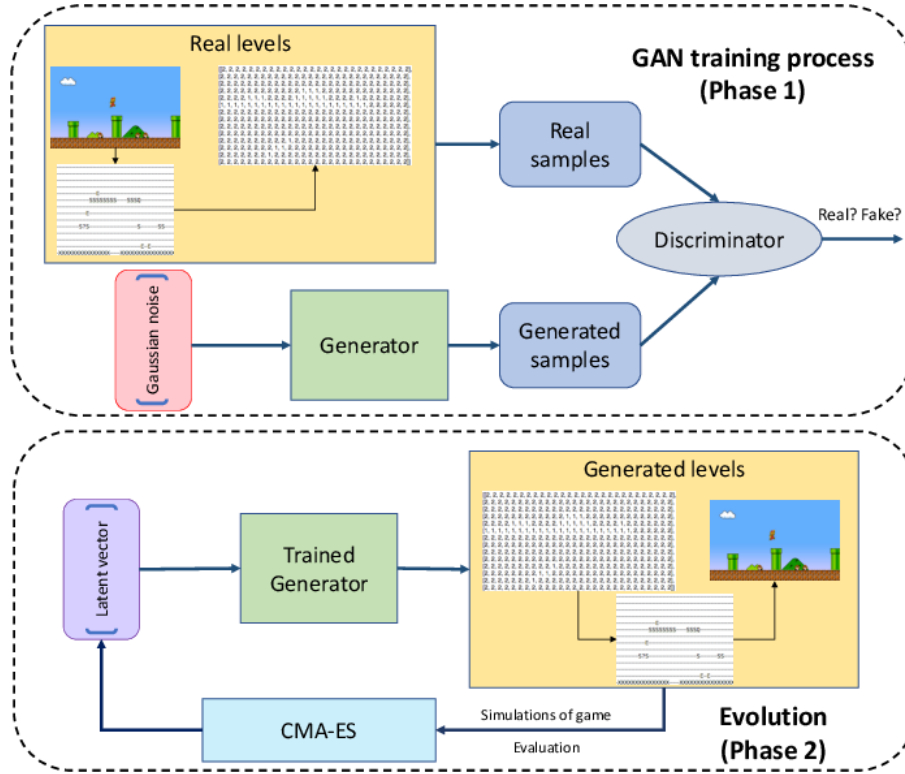


Figura 2.3: Representação do processo de treinamento e de LVE. Fonte: (Volz et al., 2018)

$$F_2 = F_1 + 0.5 \cdot (20.0 - n) \quad (2.2)$$

No artigo, a ideia apresentada é que a dificuldade aumente gradativamente conforme a progressão no eixo x, lembrando que o objetivo do jogo é chegar ao fim da fase, ou seja, no último ponto do eixo x. Para isso, os autores juntam dois níveis otimizados com F_1 e $t = 1$, um nível otimizado com F_1 e $g = 0.7$, e dois níveis otimizando F_2 com $g = 0.7$. O exemplo desta junção está na **Figura 2.4**.

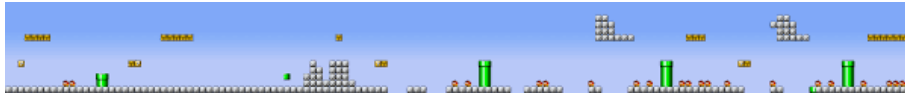


Figura 2.4: Nível gerado pelos autores, com progressão de dificuldade no eixo x. Fonte: (Volz et al., 2018)

Em outros experimentos, os autores usam duas funções relacionadas ao teste pelo agente A estrela vencedor da competição da marioAI de 2009, conforme as equações (2.3) e (2.4), onde p é a fração da fase em que o agente passou, e jumps a quantia de pulos:

$$F_3 = \begin{cases} -p, & \text{se } p < 1, \\ -p - \#jumps, & \text{se } p = 1. \end{cases} \quad (2.3)$$

$$F_4 = \begin{cases} -p + 60, & \text{se } p < 1, \\ -p + \#jumps, & \text{se } p = 1. \end{cases} \quad (2.4)$$

Na verdade, F_3 maximiza os pulos e recompensa fases passáveis, enquanto F_4 também recompensa fases jogáveis, mas minimizando pulos. A **Figura 2.5** ilustra alguns desses níveis gerados.



Figura 2.5: Nível gerado pelos autores, de acordo com a performance do agente. A letra a utiliza F_3 , enquanto a parte b utiliza F_4 . Fonte: (Volz et al., 2018)

Lembrando que não foram feitas abordagem multi-objetivo utilizando F_1 ou F_2 e F_3 ou F_4 .

2.9 CONCLUSÃO

Neste capítulo, foram apresentados os conceitos de GAN, PCGML e LVE, além de três algoritmos bioinspirados - CMA-ES, PSO e NSGA-II - que se relacionarão diretamente aos demais conceitos no decorrer do trabalho. Também foi apresentado o algoritmo A estrela, que será utilizado para teste e otimização das fases, além de uma visão ampla de um dos principais artigos da área de PCGML, artigo em que a rede proposta ficou conhecida como *MarioGAN*. No próximo capítulo, serão visitados alguns outros artigos da área, muitos os quais utilizam de alguma forma os conceitos aqui apresentados.

3 TRABALHOS RELACIONADOS

Este capítulo apresenta uma visão geral de estudos existentes em relação ao uso de redes generativas adversariais para geração de fases e também do uso da exploração de variáveis latentes para guiar o conteúdo gerado. Ele revisa a definição de geração de conteúdo procedural, apresenta sistemas utilizados para geração de conteúdo e discute modelos de geração de conteúdo relevantes para a literatura.

3.1 GERAÇÃO DE CONTEÚDO PROCEDURAL VIA MACHINE LEARNING UTILIZANDO REDES GENERATIVAS ADVERSARIAIS

Antes mesmo do machine learning chegar a área de geração procedural de conteúdo, vários métodos baseados em busca como n-gramas e cadeias de markov já eram empregados como levantado em (Togelius et al., 2011). Neste artigo de certa popularidade, PCG é definida como "criar conteúdo para jogos automaticamente, através de meios algorítmicos". Já em (Hendrikx et al., 2013), os autores definem PCG-G (geração de conteúdo procedural para jogos) como "a aplicação de computação para gerar conteúdo para jogos, distinguindo instâncias interessantes entre as amostras geradas, e selecionar instâncias interessantes em prol dos jogadores". Embora o termo não tenha se popularizado, (Hendrikx et al., 2013) comenta muito bem vários dos desafios dessa área, com ênfase em como julgar o valor técnico e cultural do conteúdo gerado.

Alguns exemplos de PCGML são o *Bardo Composer* (Ferreira et al., 2020) que gera músicas para RPGs de Mesa, geração de personagens em jogos no estilo pixel-art em (Rebouças Serpa e Formico Rodrigues, 2019), ou até mesmo aplicações mais práticas, como a desenvolvedora *CD Projekt RED*, que fez uma parceria com a empresa canadense *Jali Research* para, através de técnicas híbridas de inteligência artificial clássica e aprendizado de máquina, fazer com que a boca dos personagens se adapte a dublagem correspondente do diálogo, e o recente anúncio da desenvolvedora francesa *Ubisoft* do desenvolvimento de uma ferramenta para ajuda de construção de narrativas de jogos, a *Ubisoft Ghostwriter*.

Porém a geração de conteúdo procedural data ao jogo de 1984 *Elite*, onde os desenvolvedores usavam uma sequência definida de sementes aleatórias para gerar composições de planetas na intenção de economizar memória armazenando apenas um gerador ao invés dos planetas inteiros. De acordo com o gráfico da **Figura 2.1**, quando se fala de PCGML, há uma quantia desproporcional de pesquisa em geração de níveis 2D em comparação aos demais tópicos. Para além disso, (Liu et al., 2020) ilustra no mesmo gráfico o quanto o paradigma do aprendizado adversarial domina este tópico. Segundo os autores, o modelo adversarial mais popular neste campo seriam as GANs e suas variações.

Em (Giacomello et al., 2018), algumas GANs combinaram imagens e atributos topológicos e posicionais extraídos de conteúdo feito por humanos para gerar níveis do jogo *Doom*.

Neste trabalho em específico, os níveis não foram testados para checar se são jogáveis ou não. Já em (Torrado et al., 2019), além de propor uma nova arquitetura que acrescenta camadas de *embedding* e módulos de atenção chamada CESAGAN, a pequena quantia de dados de treino do jogo *The Legend Of Zelda (1986)* foi expandida por uma técnica de *bootstrapping*. Alguns dos resultados de (Torrado et al., 2019) estão na **Figura 3.1**.

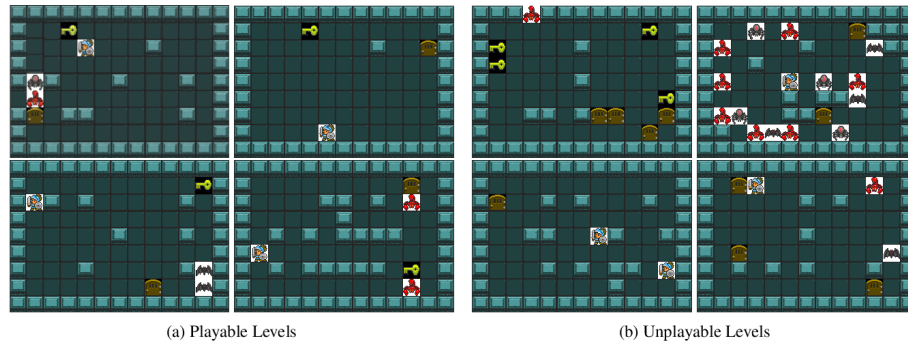


Figura 3.1: Fases geradas pela CESAGAN. Em a, níveis jogáveis e em b níveis injogáveis. Fonte: (Torrado et al., 2019)

(Capps e Schrum, 2021) utiliza múltiplas GANs, um modelo o qual os autores apelidaram como multiGAN, para gerar níveis melhores conectados do jogo *megaman* (1987). Para evoluir os níveis a atingirem certa propriedade, ao invés do CMA-ES, os autores utilizam o NSGA-II (*Non-Dominated Sorting Genetic Algorithm*), um algoritmo multi-objetivo baseado em conceitos de Pareto. Para além disso, (Capps e Schrum, 2021) utiliza um algoritmo A estrela para prever os caminhos e ajudar na mutação das variáveis latentes.

Para além de geração de níveis, GANs também são utilizadas para produzir proceduralmente outros conteúdos. (Hong et al., 2019) propõe o uso de dois codificadores (*encoder*) distintos para estrutura, formato e cor, juntamente com discriminadores distintos para formato e cor para geração de silhuetas de personagens. (Jin et al., 2017) também utiliza redes generativas adversariais para gerar personagens, desta vez para o jogo *The Sims* (2000). Em outro exemplo, (Fadaeddini et al., 2018) utiliza GANs para geração de texturas em jogos.

Como já explicado anteriormente, (Volz et al., 2018) define uma DCGAN baseada na WGAN apresentada por (?) treinada em um único nível do jogo *Super Mario Bros* e inova na forma de adicionar restrições a rede geradora após seu treinamento utilizando a técnica de LVE para adicionar restrições a saída do modelo. Neste artigo, o CMA-ES é o algoritmo utilizado para a busca no espaço latente e é aplicado após o treinamento da rede neural adversarial.

3.2 CONCLUSÃO

Neste capítulo, foram revisitados alguns trabalhos que utilizam conceitos de PCGML, muitos trabalhos estes que utilizam conceitos de (Volz et al., 2018), além de algumas aplicações práticas no início da sessão. Não se limitando a geração de níveis, o objetivo desta sessão foi justificar a importância do aprendizado adversarial na área de PCGML como um todo, área esta

bastante aquecida porém recente, sem um estado da arte definido. No próximo capítulo, será apresentada a proposta deste trabalho.

4 PROPOSTA

Este capítulo apresenta os métodos propostos para comparação entre os métodos de LVE, bem como os para avaliação do conteúdo gerado por redes treinadas em diferentes conjuntos. Ele apresenta a arquitetura do modelo neural adversarial utilizado, o conjunto de dados em que o modelo foi treinado e as funções objetivo propostas.

4.1 CONJUNTO DE TREINO

O jogo escolhido para o trabalho é o mesmo presente em (Volz et al., 2018), *Super Mario Bros.* Enquanto (Volz et al., 2018) se propõe a avaliar a diversidade de conteúdo gerado a partir do treinamento do modelo com um único nível (o primeiro), neste trabalho é avaliado o quão diversificados são os conteúdos gerados em conjuntos de granularidade diversa.

Como em (Volz et al., 2018), a representação utilizada é a mesma do repositório aberto da *Video Game Level Corpus* ((Summerville et al., 2016)), em que os blocos estão representados na **Figura 2.1**. Reproduzindo a divisão do artigo de inspiração, a representação textual é dividida por uma janela deslizante de 14 blocos/caracteres de altura e 28 blocos/caracteres de largura. Além do modelo disponibilizado por (Volz et al., 2018), treinado em uma única fase, nós treinamos redes com os oito mundos (agrupamentos de níveis com dificuldade gradual). A **Tabela 4.1** mostra, em sua totalidade, a quantidade de níveis processados em cada conjunto de treino, bem como a quantidade de janelas obtidas.

Mundo	Número de Níveis
Mundo 1	3 Níveis
Mundo 2	1 Nível
Mundo 3	2 Níveis
Mundo 4	2 Níveis
Mundo 5	2 Níveis
Mundo 6	3 Níveis
Mundo 7	1 Nível
Mundo 8	1 Nível

Tabela 4.1: Número de Níveis por Mundo

4.2 ARQUITETURA DO MODELO NEURAL

Neste trabalho, a arquitetura do modelo de (Volz et al., 2018) foi preservada, arquitetura esta devidamente ilustrada na **Figura 2.2**. A principal motivação para tal preservação é para que a comparação entre conjuntos de treinamento e métodos de busca em espaço latente seja justa. Embora seja possível facilmente trocar o modelo no fluxo de dados (assim como é possível

trocar o modelo de LVE), este não é o foco deste trabalho. O código utilizado no modelo é o mesmo disponibilizado por (Volz et al., 2018), com sua implementação com auxílio da biblioteca *PyTorch* ((Paszke et al., 2019)).

4.3 MÉTODOS PARA LVE

Este estudo realiza a comparação de três diferentes métodos para LVE. Primeiro, o modelo neural generativo adversarial é devidamente treinado em algum conjunto de níveis humanos do jogo *Super Mario Bros*. Após isso, algum dos métodos de evolução de variável latente é aplicado na intenção de guiar a geração de conteúdo para atender propriedades específicas.

Os algoritmos utilizados para a evolução de variável são: o CMA-ES, como em (Volz et al., 2018), o PSO como inovação, e o NSGA-II, que foi usado em (Torrado et al., 2019). A motivação para a escolha desses três algoritmos foi, respectivamente, manter o algoritmo utilizado em (Volz et al., 2018) juntamente com seu paradigma (um algoritmo genético), testar a otimização por enxame de partículas por ser simples e rápida, e também avaliar uma abordagem multi-objetivo. A **Tabela 4.2** sintetiza os hiperparâmetros destes três algoritmos.

Algoritmos	Hiperparâmetros
CMA-ES	$Popsiz = 28, \sigma_0 = 0.5, iterations = 50$ (Para a comparação de tempo)
PSO	$Popsiz = 28, eliminate_duplicates=True, n_gen = 50$
NSGA-II	$Popsiz = 28, eliminate_duplicates=True, n_gen = 50$

Tabela 4.2: Configurações dos Algoritmos com seus Hiperparâmetros

Para o CMA-ES, a biblioteca utilizada foi a *CMA*((Hansen et al., 2019)). Já para o PSO e o NSGA-II, foram usadas as implementações do *framework pymoo*((Blank e Deb, 2020)).

4.4 FUNÇÕES OBJETIVOS

Neste trabalho, muitas funções objetivos foram otimizadas. Foram minimizadas as funções das equações (2.1) e (2.2) seguindo a proposta de progressão de dificuldade em (Volz et al., 2018), e a performance do CMA-ES e do PSO foram comparadas. Em comparação, fizemos algumas rodadas de otimização destas funções em conjunto com a presente na equação (2.3) utilizando NSGA-II, para avaliar a performance geral.

Além das funções das equações (2.1), (2.2) e (2.3), elaboramos outra função para comparar a topologia das fases. a função objetivo autoral otimizada é relacionada ao número de blocos de um certo tipo do conteúdo gerado, seguindo a equação (4.1).

$$\frac{(100 - (\sum_{i=1}^{14} \sum_{j=1}^{28} f(x_{ij}, b)))}{100} \quad \text{tal que} \quad f(x, b) = \begin{cases} 1 & \text{se } x = b \\ 0 & \text{caso contrário} \end{cases} \quad (4.1)$$

Onde i é o valor de linha e j o da coluna da matriz que representa um nível, e x_{ij} o valor numérico presente nesta posição e b o valor numérico a ser maximizado, ou seja, a representação numérica do bloco a ser maximizado. Os valores de b utilizados foram 6 (um pedaço do cano) e 5 (número de inimigos).

4.5 AVALIAÇÃO DE CONTEÚDO

Como devidamente elicitado por (Hendrikx et al., 2013), uma das maiores dificuldades da PCGML é avaliar o conteúdo gerado proceduralmente. Neste trabalho, foram propostas três métodos de avaliação:

- Os valores das funções objetivos utilizadas na técnica de LVE;
- Um classificador para avaliar se os geradores seguem suficientemente um padrão;
- Um agente para testar a jogabilidade dos níveis.

Para que cada teste deste seja em quantidades suficientes, avaliadas 500 entradas aleatórias e 500 execuções de cada algoritmo para cada função objetivo, com exceção do NSGA-II que, por ser uma abordagem multi-objetivo, otimizou a função para $b = 5$ e $b = 6$ simultaneamente. Na totalidade, por rede, foram avaliados 3000 níveis gerados.

4.5.1 Classificação de níveis

A primeira abordagem para a classificação de níveis foi a criação de um SVC de classe única treinado em todas as janelas deslizantes geradas em cima dos níveis existentes. Uma vez que esse método mostrou-se insuficiente por classificar um pouco menos da metade do conjunto de treinamento como anomalia, foi treinado um outro SVC buscando separar os 8 mundos mais uma classe extra de matrizes 14 X 28 com números aleatórios entre 0 e 9, totalizando 9 classes. A performance deste classificador não ultrapassou 55% de acurácia.

Por fim, uma rede neural simples, codificada em *PyTorch* foi treinada em cima das mesmas janelas deslizantes e sua performance foi julgada suficiente para avaliar o padrão de uma fase. Este modelo possui apenas duas camadas convolucionais com 32 neurônios, uma camada maxpooling e uma camada *fully connected*, com 64 neurônios, finalizando com uma camada de ativação softmax de 9 classes.

Para este treinamento, 80% do conjunto de dados foi usado para treino, 10% para validação e 10% para teste. Os resultados do teste estão presentes na **Tabela 4.3**.

Para os testes de classificação, este modelo foi empregado em torno das saídas do modelo levando em conta as entradas geradas.

Mundo	Precisão	Recall	F1-Score
Lixo	1.00	1.00	1.00
Mundo 1	0.92	0.90	0.91
Mundo 2	1.00	1.00	1.00
Mundo 3	1.00	1.00	1.00
Mundo 4	1.00	1.00	1.00
Mundo 5	0.73	0.73	0.73
Mundo 6	0.96	1.00	0.98
Mundo 7	1.00	1.00	1.00
Mundo 8	1.00	0.97	0.98
Total Médio	0.95	0.95	0.95

Tabela 4.3: Resultados de Precisão, Recall e F1-Score por Mundo no teste

4.5.2 Jogabilidade e performance de jogo

Em 2009, em associação a *IEEE Games Innovation Conference* e a *IEEE Symposium on Computational Intelligence and Games*, ocorreu a *Mario AI Competition*, documentada em (Togelius et al., 2010). Até hoje, um dos agentes mais utilizados para testar a jogabilidade de níveis gerados de *Super Mario Bros* é o agente A estrela de autoria de Robin Baumgarten.

Este agente define o jogo como estado atual e leva em conta a velocidade do jogador para os estados próximos. Para além do estado próximo, a heurística leva em conta que o agente está sempre na velocidade máxima, o que a torna nem tanto admissível, sendo uma relaxação segundo o autor. Tanto em (Volz et al., 2018) quanto em (Togelius et al., 2010) é dito que o agente performa em níveis sobre-humanos, agente este que é utilizado até os dias de hoje, como em (Sudhakaran et al., 2023).

Seguindo os trabalhos já existentes, nós testamos a jogabilidade das fases com o agente de Baumgarten, assim como este agente foi utilizado para o uso da equação (2.3) nos experimentos multi-objetivo.

4.6 CONCLUSÃO

Este capítulo sintetiza a ideia e o objetivo de cada experimento proposto, apresenta o conjunto de treinamento, as funções utilizadas dentro do LVE, as comparações entre os métodos de manipulação do espaço latente, e como serão avaliados estes resultados. No próximo capítulo, os resultados serão apresentados.

5 RESULTADOS

Neste capítulo, são apresentados os resultados dos testes elicitados no capítulo anterior. Com análises tanto visual quanto em tabelas e gráficos, foram aplicadas múltiplas funções de LVE na GAN, que foi treinada com múltiplos mundos. Também vale lembrar que três tipos de algoritmos foram utilizados (CMA-ES, PSO e NSGA-II) e, além das funções objetivo, os níveis gerados também foram avaliados por um classificador neural simples e o algoritmo A estrela de Robin Baumgarten.

5.1 COMPARAÇÃO DE TEMPO E PERFORMANCE ENTRE CMA-ES E PSO

Por ser simples, utilizando apenas algumas fórmulas, uma comparação entre o PSO e o CMA-ES nos quesitos tempo e performance. Embora, em sua essência, o CMA-ES não seja limitado em iterações mas sim em um valor aceitável da função objetivo, para esta comparação de tempo ele foi limitado em 50 iterações, assim como o algoritmo de partículas. Neste experimento, o conjunto de treino do gerador foi composto apenas por janelas geradas pelo processamento da primeira fase, para devidamente comparar o CMA-ES manipulando um gerador treinado com o conjunto de treino presente no artigo e sua implementação com o PSO.

A função objetivo otimizada foi a da equação 2.2, com $g = 5$. Como nos experimentos dos autores duas janelas diferentes minimizavam a função desta equação, foram realizadas 200 iterações por método de LVE. Os resultados estão presentes na **Tabela 5.1**, com média e desvio padrão.

Método	Média da Fitness	Desvio Padrão (Fitness)	Média do Tempo	Desvio Padrão (Tempo)
CMA-ES	10.0	0	8.50	2.69
PSO	10.2	0.143	2.08	0.615

Tabela 5.1: Comparação de Tempo e Fitness do CMA-ES e PSO

Os resultados são interessantes. Primeiro, vale notar que o CMA-ES obteve um desvio padrão de 0, com todas suas execuções convergindo a uma região muito parecida. Em caso de dúvida, algumas destas fases foram checadas e, de fato, as entradas - assim como as fases geradas - são diferentes, sendo pontos do espaço latente razoavelmente distantes entre si. Mas mais interessante que isso é notar como é pequena a diferença, algo que apenas um teste estatístico poderia dizer com toda certeza se é significativa ou não. Fato é que cada execução do CMA-ES levou, em média, 8.5 segundos, enquanto as execuções do PSO levaram apenas 2.08 segundos. Isto leva a crer que o uso de um algoritmo mais simples, como o de otimização de partículas, é relevantemente mais rápido que o CMA-ES, e alcança marcas parecidas ou até mesmo superiores.

5.2 NÍVEIS EM PROGRESSÃO DE DIFICULDADE

Neste experimento, nós comparamos o desempenho e a variedade de fases geradas dos métodos de LVE CMA-ES e PSO utilizando as funções das equações (2.1) e (2.2) da mesma forma sugerida no artigo: concatenando duas janelas geradas minimizando a função da equação (2.1) com $t = 1$, uma com $t = 0.7$, e outras duas utilizando a equação (2.2) com $t = 0.7$. Neste experimento, foram considerados três diferentes conjuntos de treino: o conjunto gerado pelo processamento apenas da primeira fase (o mesmo de (Volz et al., 2018)) e os gerado pelo processamento das três fases do mundo 1 e 6. Para cada uma das cinco partes foram geradas 100 instâncias, totalizando 500 fases por método de LVE e fases, totalizando 13500 níveis analisados, com 300 para cada pedaço das fases. A **tabela 5.2** mostra os resultados desta comparação. (tabela em construção)

Em cons- trução
Em constru- ção

Tabela 5.2: No Caption

(NOTA: Depois de analisar a tabela, vou colocar alguns níveis de forma visual)

5.3 TOPOLOGIA DOS NÍVEIS

Assim como foram analisadas as funções propostas em (Volz et al., 2018), também foi observado o efeito da evolução de variáveis latentes com uma função diretamente ligada com a topologia das fases, no caso a função da equação (4.1) que busca maximizar um certo tipo de bloco, isto é, a quantia de vezes que um elemento (ou número, em termos literais) aparece na matriz que representa um nível. Para este experimento o bloco maximizado, ou seja, o valor de b na equação (4.1), foi 5 (inimigos) e 6 (canto superior esquerdo do cano).

Para além da análise de um modelo treinado com o mesmo conjunto de (Volz et al., 2018), ou seja, apenas com a primeira fase do jogo *Super Mario Bros*, também foram feitos treinamentos com cada um dos conjuntos presentes na **Tabela 4.1**, ou seja, com cada um dos conjuntos de fases disponíveis agrupados em mundo. Para cada conjunto foram realizadas 1000 iterações do PSO e do CMA-ES, sendo 500 para maximizar inimigos e 500 para maximizar canos, enquanto o NSGA-II realizou apenas 500 iterações, otimizando simultaneamente a função para $b = 5$ e $b = 6$. Adicionalmente, foram realizadas 500 iterações com entradas aleatórias por conjunto de treino, na intenção de comparar os efeitos do uso da LVE com entradas ruidosas sem cuidado algum. Em todos os casos, as entradas foram limitadas a números reais entre -2 e 2. Os resultados estão nas **tabela 5.3** e **tabela 5.4**.

As **tabelas 5.3** e **5.4** contém vários resultados interessantes. O mais importante deles é que para qualquer método usado para a LVE - seja o CMA-ES, o PSO ou o NSGA-II - a

função objetivo otimizada e, conseqüentemente, o tipo de elemento visado (maximizando canos na primeira tabela e inimigos na segunda) aparece em quantia maior do que quando comparado com entradas aleatórias, significando que o uso da LVE possui sim impacto relevante pelo menos quando se diz respeito à topologia da fase. Para além disso, na grande maioria das fases geradas a partir de múltiplos conjuntos de treino, o CMA-ES é o algoritmo que melhor converge. Porém vale lembrar que, além do tempo de execução do CMA-ES ser maior que o do PSO, o CMA-ES neste experimento não foi limitado em número de gerações, e também a diferença entre os valores da média das funções objetivo do PSO e o CMA-ES ainda é pequena e, caso algum teste estatístico corrobore que não são significativas, em aplicações práticas em que o tempo de geração deveria ser menor o PSO seria a melhor opção, mesmo com valores um pouco abaixo do CMA-ES.

Em relação ao NSGA-II, embora ele não tenha superado no objetivo principal nenhum dos outros métodos de LVE, no oposto (número de inimigos quando se maximiza canos ou número de canos quando se maximizando canos) ele sempre é superior com exceção do mundo 6, o que poderia levar a um estudo da correlação entre o número de canos e de inimigos neste mundo específico. Além disso, como já dito anteriormente, o NSGA-II ainda foi melhor que as entradas aleatórias, justificando seu uso numa situação de equilíbrio entre duas funções não correlacionadas em relação à topologia do nível. Outro detalhe que deve se ter atenção é que, para as análises do NSGA-II foi utilizado um critério de desempate que equilibra as duas funções igualmente, e o algoritmo pode ou não ter gerado soluções não dominadas mais interessantes que os demais.

5.3.1 Análise visual

Para além de resultados métricos, o fato de o que estar sendo analisado ser as fases do jogo, podemos analisar de forma visual os níveis gerados. Alguns casos interessantes foram selecionados. A **Figura 5.1** mostra 10 conteúdos gerados por uma GAN treinada com as amostras do mundo dois, de forma aleatória, algumas sendo não passáveis, outras com alguma quantidade de canos e inimigos. De imediato, podemos afirmar que, visualmente, os níveis são diferentes com alguns graus de semelhança. A **Figura 5.2** mostra a mesma GAN, mas utilizando o CMA-ES como técnica de LVE e maximizando canos. Analisando estes níveis, podemos ver que a geração de canos foi, de fato, priorizada, com algumas ressalvas. Nesta função, o bloco que corresponde ao número 6 da matriz faz contar como o elemento, mas, na prática, o que queremos é mais canos. Em alguns dos níveis da **Figura 5.2** há canos sem o canto superior esquerdo, não contando para a função, assim como há canos quebrados que possuem mais de um bloco deste tipo, sendo que no conjunto de treino cada cano possui apenas um desses blocos. Isto nos leva a crer que o contexto não foi totalmente traduzido, sendo talvez ainda mais ignorado com métodos de LVE.

Já num exemplo utilizando o mundo 5 como conjunto de treinamento, que tem fases mais planas, podemos ver como se comporta a função maximizando o número de inimigos. A **figura 5.3** mostra conteúdo aleatórios gerados pela GAN treinada com o mundo 5, já as **figura**

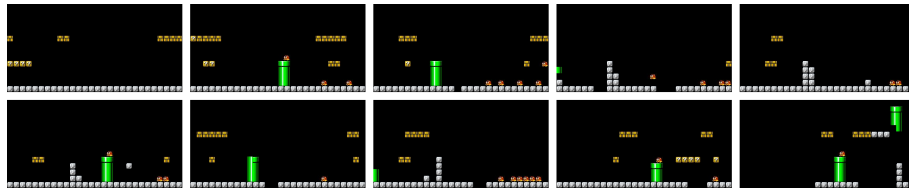


Figura 5.1: Fases aleatórias geradas pela GAN treinada com o mundo 2

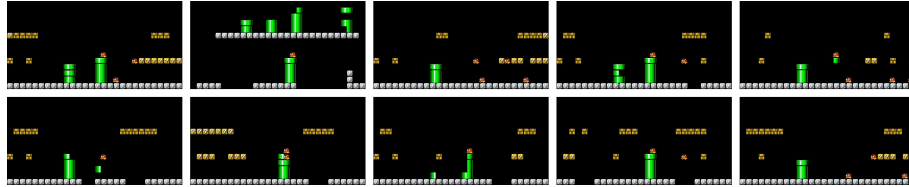


Figura 5.2: Fases geradas pela GAN treinada com o mundo 2 utilizando o CMA-ES como técnica de LVE, maximizando canos

5.4 e 5.5 mostram níveis utilizando o CMA-ES e o PSO como técnicas de LVE, respectivamente, aplicando a equação (4.1) como função, maximizando o número de inimigos. Por fim, a **figura 5.6** mostra níveis gerados pelo NSGA-II. Visualmente, podemos afirmar que o PSO e o CMA-ES conduziram o espaço latente de forma a gerarem níveis razoavelmente parecidos. Já o NSGA-II conseguiu gerar alguns canos além dos inimigos, porém, não em uma quantia exorbitante, já que não faz sentido gerar algo que você não vê tanto em seu conjunto de treino.

Por fim, outro caso interessante é utilizar a LVE maximizando inimigos conhecendo o mundo 6 como conjunto de treino, conforme mostra a **figura 5.7**, em que a técnica de LVE utilizada foi o PSO. Neste caso, não há inimigos pois as fases do mundo 6 não possuem inimigos, são focadas em pulos complexos - e as simplificações do dataset da VGLC ignoram alguns outros elementos - mostrando que, se de fato não conhece aquilo, fica impossível gerar tal conteúdo.

5.4 CLASSIFICAÇÃO DOS PADRÕES GERADOS

É importante também avaliarmos o conteúdo gerado. Como um SVC de uma classe ou múltiplas não se mostraram suficientes para dividir as janelas deslizantes dos mundos, um modelo neural simples foi testado e, como mostrado no capítulo anterior, alcançou valores satisfatórios. Uma vez decidida a forma de classificar o padrão, esta rede foi utilizada para avaliar os pedaços de fase gerados, comparando as entradas geradas de forma aleatória e utilizando o PSO e o CMA-ES como técnicas de evolução de variáveis latentes. Nesta comparação, nós utilizamos nove conjuntos diferentes de treino da rede geradora, sendo a primeira fase como em (Volz et al., 2018) e cada um dos oito mundos, totalizando 9 conjuntos de treino. Para cada conjunto, foram geradas 500 entradas aleatórias, 500 entradas com uso de CMA-ES como método de LVE e 500 com o uso do PSO, totalizando 1500 entradas por conjunto de treino, totalizando 1350 níveis avaliados. Neste experimento, a função otimizada foi a da equação (4.1) com $b = 6$, ou seja, maximizando o número de canos. Os gráficos presente nas figuras **5.8**, **5.9** e **5.10** sintetizam

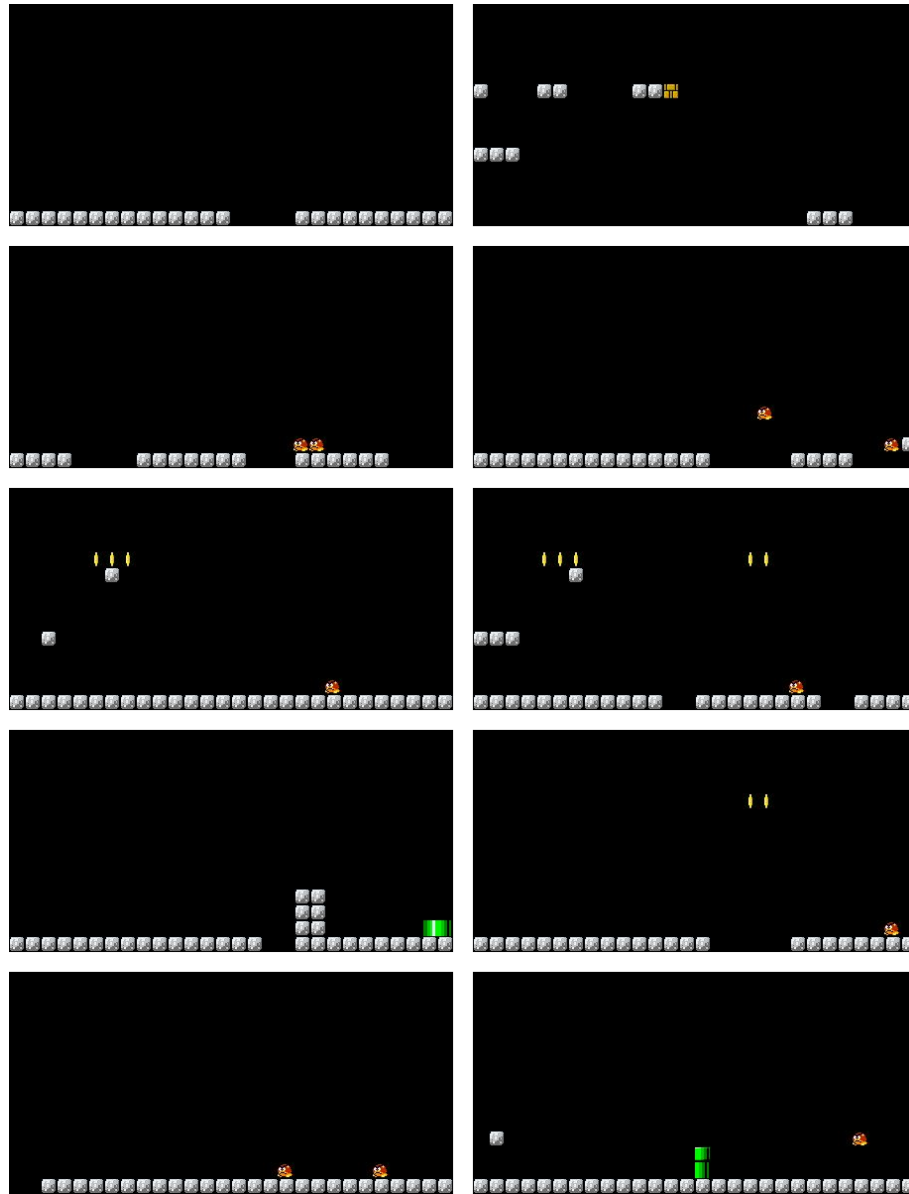


Figura 5.3: Fases aleatórias geradas pela GAN treinada com o mundo 5

estes resultados, com uso de entradas aleatórias, entradas construídas pelo CMA-ES e PSO, respectivamente.

Para além desses gráficos, é interessante ressaltar que majoritariamente as fases foram classificadas dentro de um padrão específico pela GAN. A **Tabela 5.5** explicita os padrões de cada conjunto de treino, técnica e padrão.

Embora alguns mundos do primeiro ao quinto, com exceção do quarto, sejam classificados fora do padrão esperado, é importante ressaltar que eles não distoam na questão do padrão atribuído. A GAN treinada com o mundo 3, por exemplo, tem seus conteúdo gerados majoritariamente classificados como níveis do mundo 5, independente do tipo de entrada. Comparando as entradas aleatórias com técnicas de LVE, a afirmação de que o uso de uma técnica de manipulação de espaço latente não interfere no padrão que se encaixa um conteúdo gerado ganha

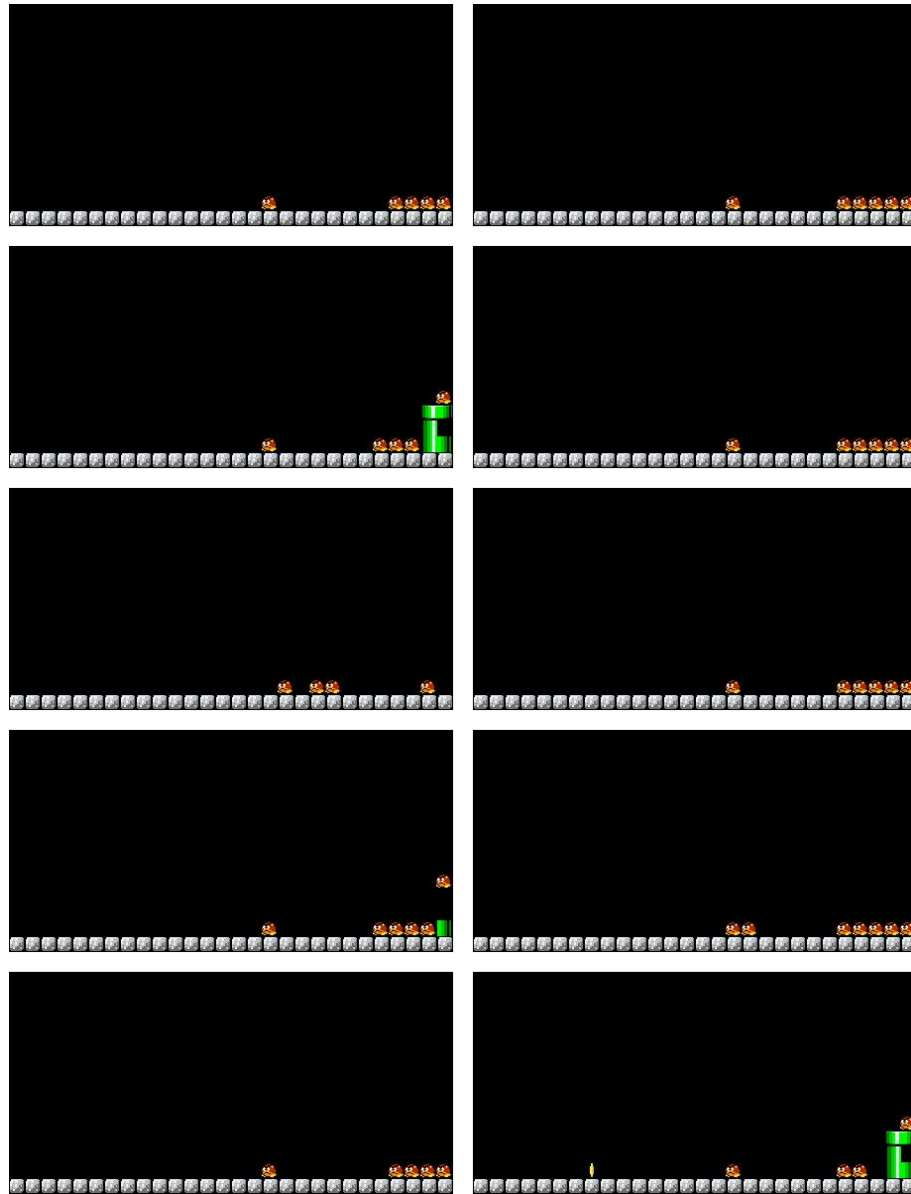


Figura 5.4: Fases geradas pela GAN treinada com o mundo 5 utilizando o CMA-ES como técnica de LVE, maximizando inimigos

um tanto de força. Porém, embora empiricamente provada com uma única função, esta possível conclusão só seria correta caso haja uma prova matemática para tal.

Para tanto, esta sessão do capítulo ainda justifica a escolha das redes treinadas com os mundos 1 e 6, que possuem mais níveis e são classificadas fora e dentro do padrão esperado, respectivamente, além da primeira fase, que é o mesmo conjunto de treinamento de (Volz et al., 2018). Com estes três conjuntos de treinos, podemos cobrir de forma satisfatória naturezas distintas de conjuntos de treino.

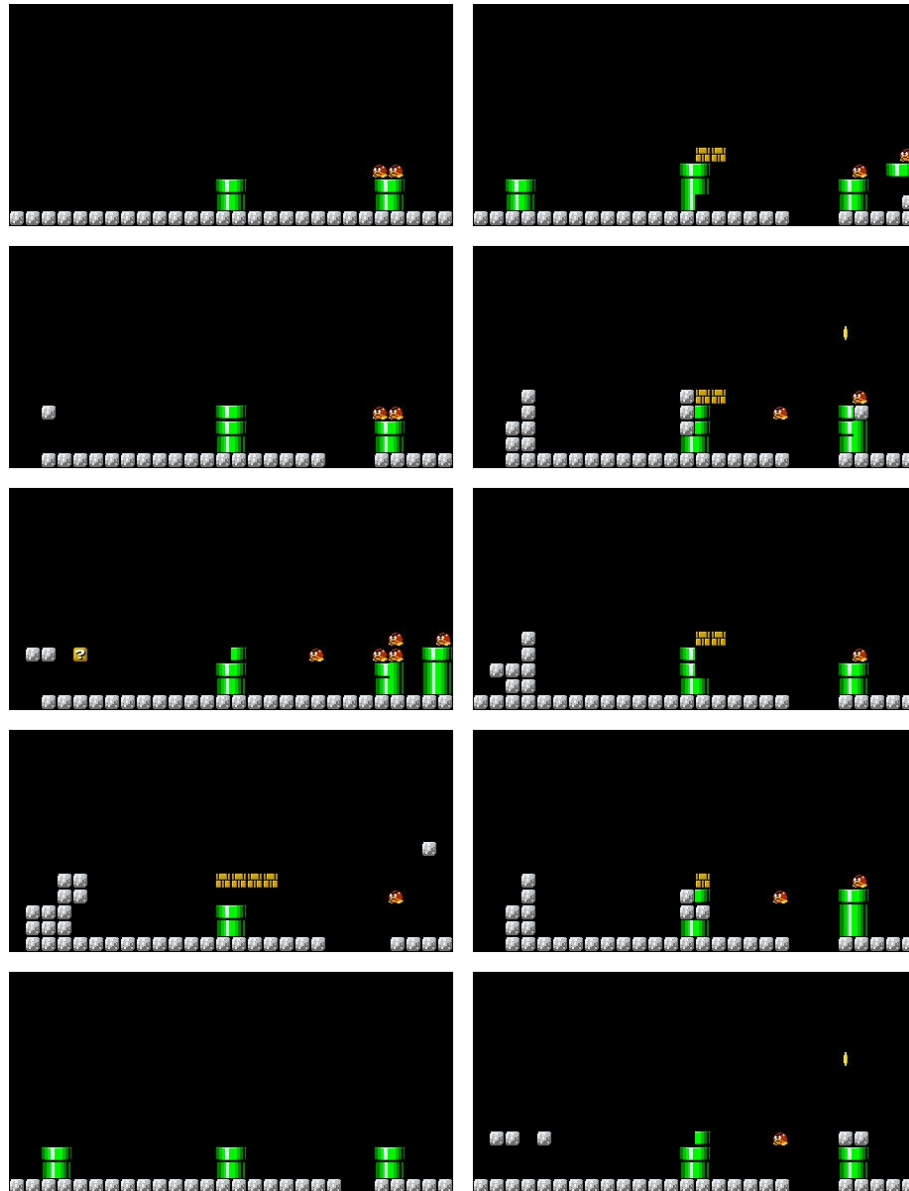


Figura 5.5: Fases geradas pela GAN treinada com o mundo 5 utilizando o PSO como técnica de LVE, maximizando inimigos

5.5 JOGABILIDADE

Além das duas formas de avaliação já apresentadas, nós avaliamos alguns conjuntos de conteúdos gerados através do agente A estrela de Robin Baumgarten além da abordagem multi-objetivo.

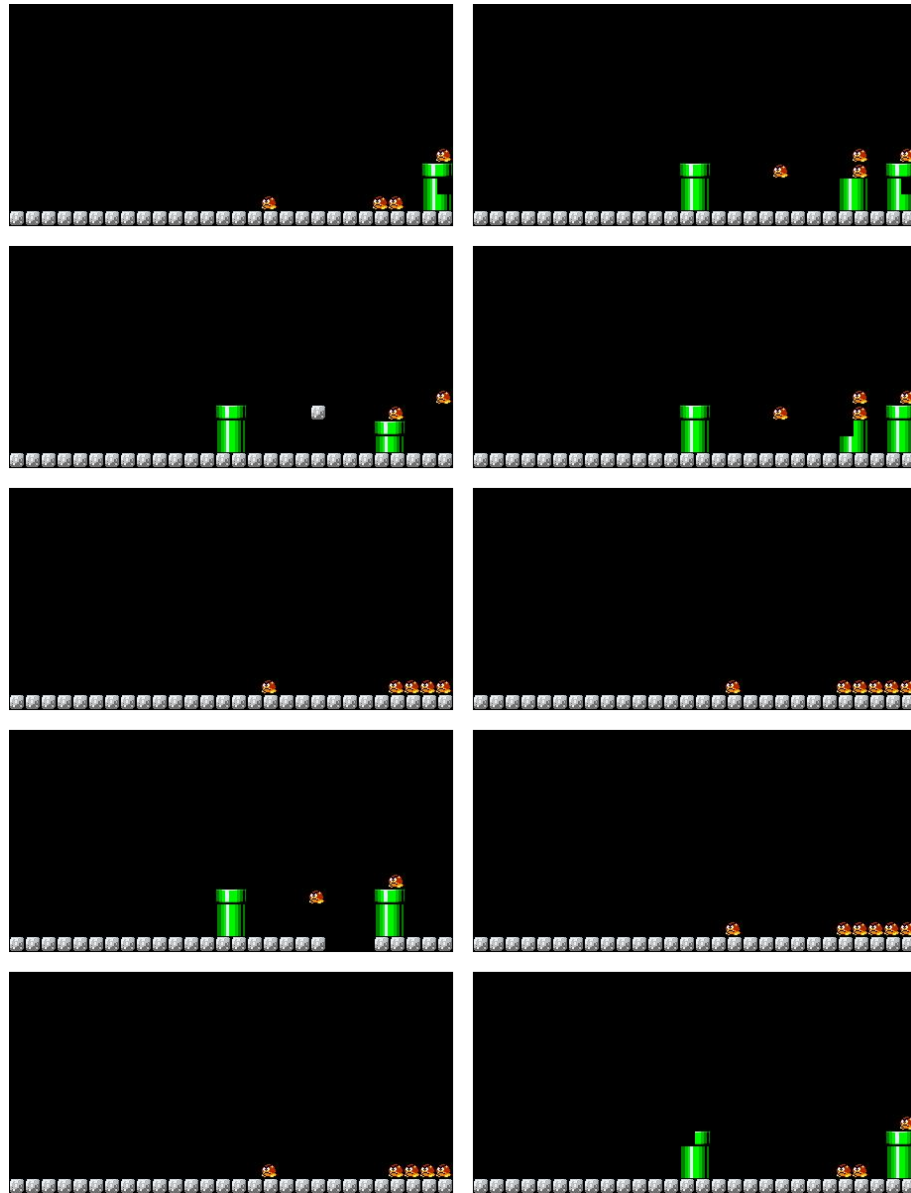


Figura 5.6: Fases geradas pela GAN treinada com o mundo 5 utilizando o NSGA-II como técnica de LVE, maximizando canos e inimigos simultâneamente

Mundo	Entrada	Média de Canos ($\bar{x}$)	Desvio Padrão (Canos) (σ)	Média de Inimigos ($\bar{x}$)	Desvio Padrão (Inimigos) (σ)
Mundo 1	Aleatórias	0.482	0.715	1.77	1.57
	CMA-ES	5.81	0.558	1.17	0.522
	NSGA-II	1.44	0.807	6.00	1.85
	PSO	4.64	0.934	1.54	1.06
Mundo 2	Aleatórias	0.650	0.548	4.07	2.68
	CMA-ES	3.43	1.12	4.61	2.26
	NSGA-II	1.01	0.498	9.27	1.78
	PSO	3.25	0.882	4.74	2.32
Mundo 3	Aleatórias	0.280	0.512	2.29	1.83
	CMA-ES	2.86	0.345	3.91	1.51
	NSGA-II	1.51	0.686	7.17	1.83
	PSO	2.82	0.383	3.87	1.62
Mundo 4	Aleatórias	0.652	0.832	0.918	1.05
	CMA-ES	5.68	0.467	4.39	1.02
	NSGA-II	3.96	0.778	5.19	1.01
	PSO	5.31	0.640	4.40	1.01
Mundo 5	Aleatórias	0.230	0.527	3.05	2.14
	CMA-ES	4.08	0.599	3.41	1.38
	NSGA-II	0.530	0.809	10.7	2.21
	PSO	3.69	0.637	3.51	1.36
Mundo 6	Aleatórias	0.0900	0.293	0.0940	0.312
	CMA-ES	1.84	0.543	1.34	0.724
	NSGA-II	1.27	0.536	1.28	0.547
	PSO	1.99	0.0999	1.57	0.612
Mundo 7	Aleatórias	0.452	0.733	1.26	1.14
	CMA-ES	4.84	0.494	2.47	0.728
	NSGA-II	2.38	1.02	4.50	1.17
	PSO	4.62	0.705	2.50	0.758
Mundo 8	Aleatórias	0.778	0.922	3.56	1.96
	CMA-ES	5.85	0.398	3.91	0.870
	NSGA-II	1.64	1.01	9.86	2.05
	PSO	5.22	0.702	3.93	1.37

Tabela 5.3: Estatísticas dos conjuntos de treino maximizando o número de canos

Mundo	Entrada	Média de Canos ($\bar{x}$)	Desvio Padrão (Canos) (σ)	Média de Inimigos ($\bar{x}$)	Desvio Padrão (Inimigos)
Mundo 1	Aleatórias	0.482	0.715	1.77	1.57
	CMA-ES	0.140	0.391	11.0	0.79
	NSGA-II	1.44	0.807	6.00	1.85
	PSO	0.234	0.442	10.2	1.215
Mundo 2	Aleatórias	0.650	0.548	4.07	2.63
	CMA-ES	0.556	0.536	15.3	1.22
	NSGA-II	1.01	0.498	9.27	1.77
	PSO	0.712	0.574	13.8	1.14
Mundo 3	Aleatórias	0.280	0.512	2.29	1.830
	CMA-ES	0.806	0.519	12.2	1.03
	NSGA-II	1.51	0.686	7.17	1.82
	PSO	0.720	0.585	11.1	1.14
Mundo 4	Aleatórias	0.652	0.832	0.918	1.058
	CMA-ES	3.35	0.844	7.00	0.07
	NSGA-II	3.96	0.778	5.19	1.01
	PSO	2.97	1.07	6.80	0.449
Mundo 5	Aleatórias	0.230	0.527	3.05	2.14
	CMA-ES	0	0	15.2	1.16
	NSGA-II	0.530	0.809	10.7	2.21
	PSO	0.0380	0.270	13.5	1.52
Mundo 6	Aleatórias	0.0900	0.293	0.0940	0.31
	CMA-ES	1.27	0.673	1.86	0.62
	NSGA-II	1.27	0.536	1.28	0.54
	PSO	1.38	0.556	2.06	0.23
Mundo 7	Aleatórias	0.452	0.733	1.26	1.13
	CMA-ES	1.47	0.647	6.78	0.562
	NSGA-II	2.38	1.02	4.50	1.16
	PSO	1.14	0.697	6.43	0.63
Mundo 8	Aleatórias	0.778	0.922	3.56	1.96
	CMA-ES	1.14	0.917	16.4	0.928
	NSGA-II	1.64	1.01	9.86	2.05
	PSO	0.906	0.977	14.3	1.60

Tabela 5.4: Estatísticas dos conjuntos de treino maximizando o número de inimigos

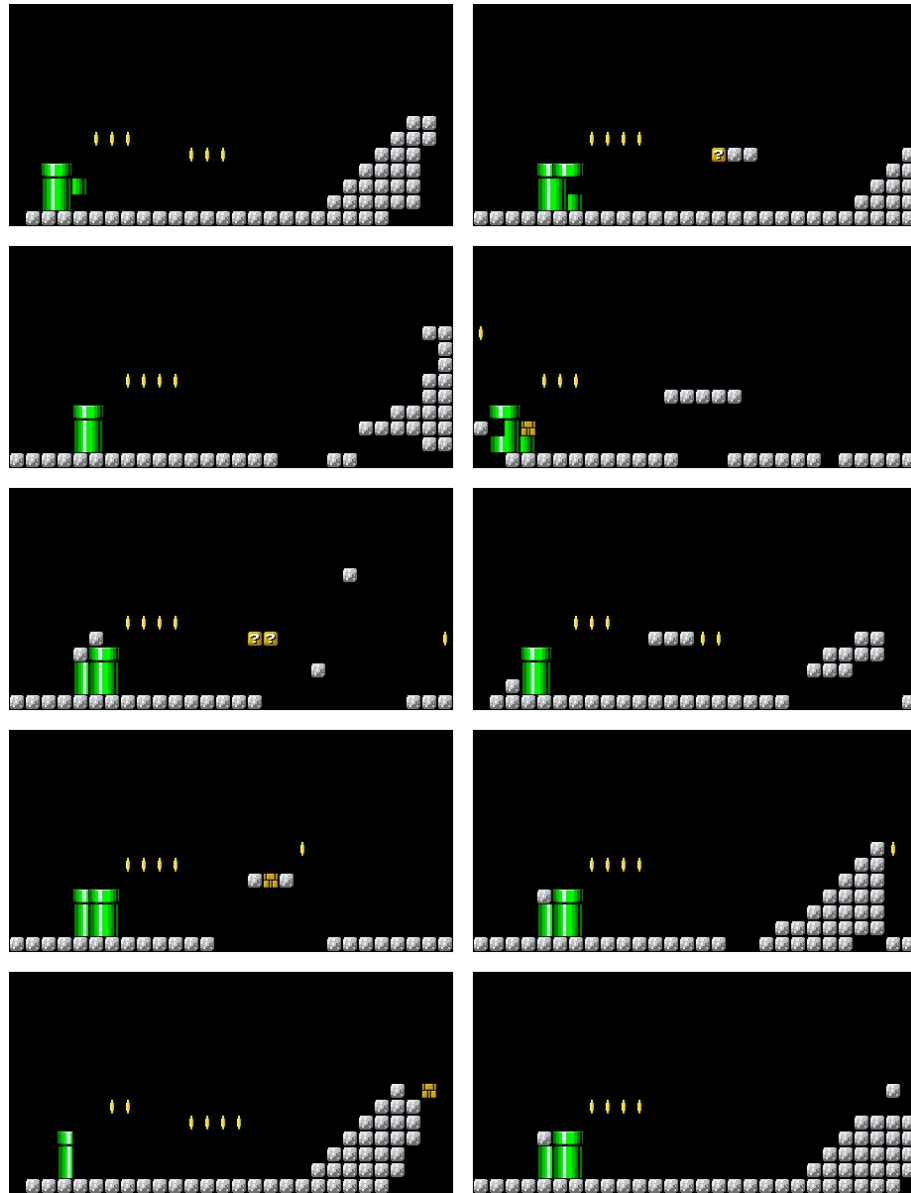


Figura 5.7: Fases geradas pela GAN treinada com o mundo 6 utilizando o CMA-ES como técnica de LVE, maximizando inimigos

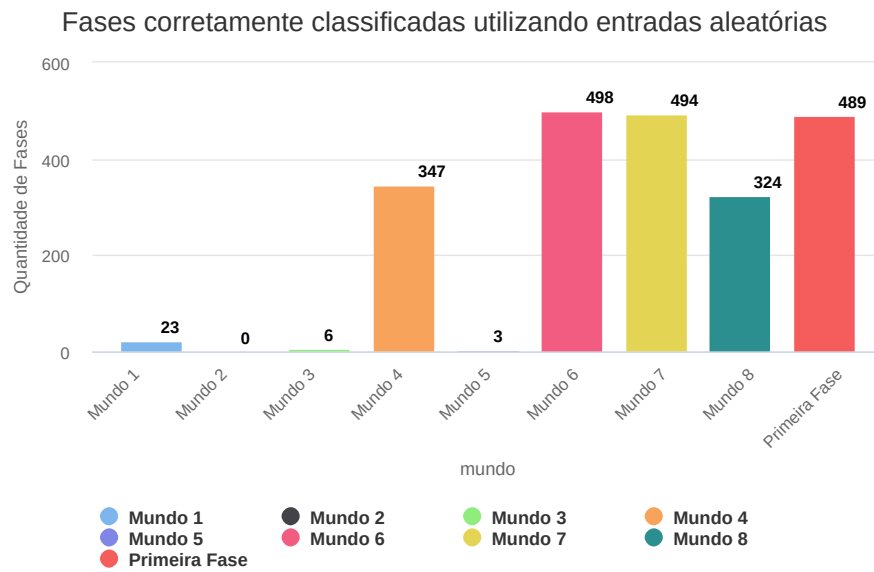


Figura 5.8: Classificação corretas das entradas aleatórias

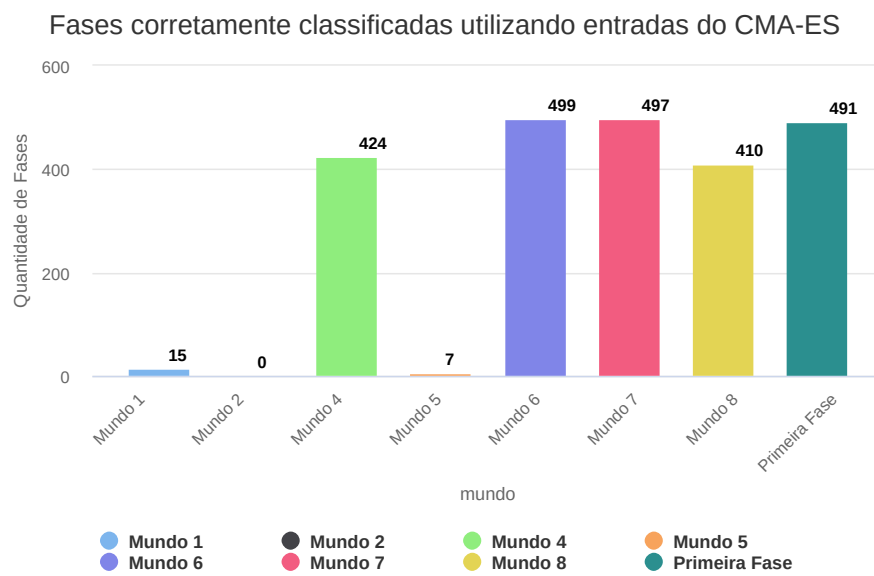


Figura 5.9: Classificação corretas das entradas do CMA-ES

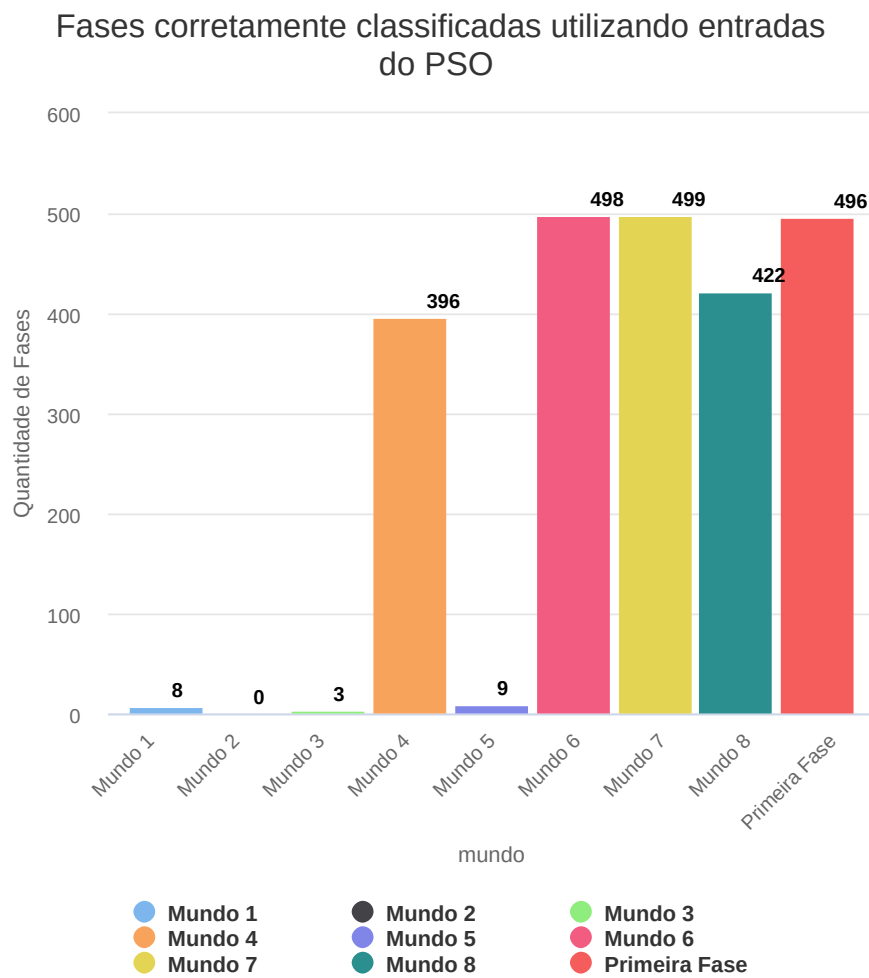


Figura 5.10: Classificação corretas das entradas do PSO

Conjunto de treino	Entradas	Classe Predominante	Quantia Classificada
Primeira fase	Aleatória	1	489
	CMA-ES	1	491
	PSO	1	496
Mundo 1	Aleatória	7	454
	CMA-ES	7	460
	PSO	7	475
Mundo 2	Aleatória	8	382
	CMA-ES	8	412
	PSO	8	431
Mundo 3	Aleatória	5	456
	CMA-ES	5	489
	PSO	5	481
Mundo 4	Aleatória	4	347
	CMA-ES	4	424
	PSO	4	396
Mundo 5	Aleatória	7	464
	CMA-ES	7	472
	PSO	7	468
Mundo 6	Aleatória	6	498
	CMA-ES	6	499
	PSO	6	498
Mundo 7	Aleatória	7	494
	CMA-ES	7	497
	PSO	7	499
Mundo 8	Aleatória	8	324
	CMA-ES	8	410
	PSO	8	422

Tabela 5.5: Descrição do Conjunto de Treino

6 CONCLUSÃO

REFERÊNCIAS

- Arjovsky, M., Chintala, S. e Bottou, L. (2017). Wasserstein gan.
- Blank, J. e Deb, K. (2020). pymoo: Multi-objective optimization in python. *IEEE Access*, 8:89497–89509.
- Bontrager, P., Togelius, J. e Memon, N. (2017). Deepmasterprint: Generating fingerprints for presentation attacks.
- Capps, B. e Schrum, J. (2021). Using multiple generative adversarial networks to build better-connected levels for mega man.
- Deb, K. (2002). A fast and elitist multiobjective genetic algorithm: Nsga-ii. Em IEEE, editor, *IEEE Transactions on evolutionary computation*, Vol. 6.
- Fadaeddini, A., Majidi, B. e Eshghi, M. (2018). A case study of generative adversarial networks for procedural synthesis of original textures in video games. Em *2018 2nd National and 1st International Digital Games Research Conference: Trends, Technologies, and Applications (DGRC)*, páginas 118–122.
- Ferreira, L. N., Lelis, L. H. S. e Whitehead, J. (2020). Computer-generated music for tabletop role-playing games.
- Giacomello, E., Lanzi, P. L. e Loiacono, D. (2018). Doom level generation using generative adversarial networks.
- Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A. e Bengio, Y. (2014). Generative adversarial networks. *Advances in Neural Information Processing Systems*, 3.
- Hansen, N., Akimoto, Y. e Baudis, P. (2019). CMA-ES/pycma on Github. Zenodo, DOI:10.5281/zenodo.2559634.
- Hansen, N., Müller, S. D. e Koumoutsakos, P. (2003). Reducing the time complexity of the derandomized evolution strategy with covariance matrix adaptation (cma-es). *Evolutionary Computation*, 11(1):1–18.
- Hendrikx, M., Meijer, S., Van Der Velden, J. e Iosup, A. (2013). Procedural content generation for games: A survey. *ACM Trans. Multimedia Comput. Commun. Appl.*, 9(1).
- Hong, S., Kim, S. e and, S. K. (2019). Game sprite generator using a multi discriminator gan. *KSH Transactions on Internet and Information Systems*, 13(8):4255–4269.

- Jin, Y., Zhang, J., Li, M., Tian, Y., Zhu, H. e Fang, Z. (2017). Towards the automatic anime characters creation with generative adversarial networks.
- Kennedy, J. e Eberhart, R. (1995). Particle swarm optimization. Em *Proceedings of ICNN'95 - International Conference on Neural Networks*, volume 4, páginas 1942–1948 vol.4.
- Liu, J., Snodgrass, S., Khalifa, A., Risi, S., Yannakakis, G. N. e Togelius, J. (2020). Deep learning for procedural content generation. *Neural Computing and Applications*, 33(1):19–37.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J. e Chintala, S. (2019). Pytorch: An imperative style, high-performance deep learning library. Em *Advances in Neural Information Processing Systems 32*, páginas 8024–8035. Curran Associates, Inc.
- Rao, S. (2010). A multiobjective genetic algorithm for feature selection in data mining.
- Rebouças Serpa, Y. e Formico Rodrigues, M. A. (2019). Towards machine-learning assisted asset generation for games: A study on pixel art sprite sheets. Em *2019 18th Brazilian Symposium on Computer Games and Digital Entertainment (SBGames)*, páginas 182–191.
- Reed, S., Akata, Z., Yan, X., Logeswaran, L., Schiele, B. e Lee, H. (2016). Generative adversarial text to image synthesis.
- Ringuet, J. L. (1992). *Compromise Programming*, páginas 51–59. Springer US, Boston, MA.
- Sudhakaran, S., González-Duque, M., Glanois, C., Freiburger, M., Najarro, E. e Risi, S. (2023). Mariogpt: Open-ended text2level generation through large language models.
- Summerville, A., Snodgrass, S., Guzdial, M., Holmgård, C., Hoover, A. K., Isaksen, A., Nealen, A. e Togelius, J. (2018). Procedural content generation via machine learning (pcgml).
- Summerville, A. J., Snodgrass, S., Mateas, M. e n'on Villar, S. O. (2016). The vglc: The video game level corpus. *Proceedings of the 7th Workshop on Procedural Content Generation*.
- Togelius, J., Karakovskiy, S. e Baumgarten, R. (2010). The 2009 mario ai competition. páginas 1 – 8.
- Togelius, J., Yannakakis, G. N., Stanley, K. O. e Browne, C. (2011). Search-based procedural content generation: A taxonomy and survey. *IEEE Transactions on Computational Intelligence and AI in Games*, 3(3):172–186.
- Torrado, R. R., Khalifa, A., Green, M. C., Justesen, N., Risi, S. e Togelius, J. (2019). Bootstrapping conditional gans for video game level generation.

- Volz, V., Schrum, J., Liu, J., Lucas, S. M., Smith, A. M. e Risi, S. (2018). Evolving mario levels in the latent space of a deep convolutional generative adversarial network. Em *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO 2018)*, New York, NY, USA. ACM.
- Zhu, J.-Y., Park, T., Isola, P. e Efros, A. A. (2020). Unpaired image-to-image translation using cycle-consistent adversarial networks.